

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	CSA1205	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)
Weightage:	30%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	SHAIK HAZIPEERA	Reg.No.:	192572056
Department:	CSE(AI)	Date:	

ANNEXURE A
Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.
2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.
3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.
4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.
5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Code of Conduct:

I SHAIK HAZIPEERA (192572056) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

S. Hazipeera

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	Algorithm Design (30%)	Use of Control Structures (20%)	Pseudocode Structure (10%)	Completeness (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

I/O System Pseudocode Design — Answers

Computer Organization & Architecture — Long Answer Set (8–10 Marks each)

Q1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.

Approach

The CPU only configures the DMA controller (source, destination, count) and starts it; the DMAC then transfers data directly to memory. The CPU is interrupted once at completion, avoiding per-word polling.

```
BEGIN DMA_Transfer(device, memAddr, byteCount)
    // 1. CPU sets up DMA controller (one-time cost)
    DMAC.sourceAddr  = device.bufferAddr
    DMAC.destAddr    = memAddr
    DMAC.transferSize = byteCount
    DMAC.mode        = BLOCK_TRANSFER
    DMAC.interruptOnComplete = TRUE

    DMAC.START()           // CPU is now free
    CPU.resumeOtherTasks()

    // 2. DMAC works independently, cycle by cycle
    WHILE DMAC.bytesRemaining > 0 DO
        DMAC.requestBus()
        DMAC.transferWord(device -> memory)
        DMAC.bytesRemaining -= wordSize
        DMAC.releaseBus()    // bus given back between words
    END WHILE

    DMAC.raiseInterrupt(TRANSFER_COMPLETE)
END

ON INTERRUPT(TRANSFER_COMPLETE):
    ISR_DMA_Complete():
        status = DMAC.readStatus()
        IF status == ERROR THEN
            handleDMAError()
        ELSE
            markBufferReady(memAddr)
        END IF
        DMAC.clearInterrupt()
        RETURN FROM INTERRUPT
```

CPU involvement is limited to setup and a single completion ISR; all data movement cycles are handled by the DMAC.

[8-10 Marks]

Q2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.

Approach

Each device has a fixed priority and its own interrupt vector. The controller always dispatches the highest-priority pending interrupt first, and higher-priority interrupts can preempt a lower-priority ISR.

```
// Vector table: index = device vector, value = ISR address
VECTOR_TABLE[0..N] = { ISR_Timer, ISR_Disk, ISR_Network, ISR_Keyboard, ... }
PRIORITY[0..N]      = priority level assigned to each device (0 = highest)

ON INTERRUPT_SIGNAL(vector):
    pendingQueue.insert(vector, PRIORITY[vector])
    INTERRUPT_DISPATCHER()

PROCEDURE INTERRUPT_DISPATCHER():
    WHILE pendingQueue is NOT EMPTY DO
        current = pendingQueue.popHighestPriority()

        IF CPU.isServicing() AND
           PRIORITY[current] < PRIORITY[CPU.currentVector] THEN
            CPU.saveContext(CPU.currentVector) // preempt lower-priority ISR
        END IF

        CPU.saveContext(current)
        CPU.currentVector = current
        ISR = VECTOR_TABLE[current]
        CALL ISR() // direct jump, no polling
        CPU.sendEOI(current)
        CPU.restoreContext() // resume preempted/normal task
    END WHILE
END PROCEDURE
```

Using a per-device vector avoids searching for the interrupt source, while the priority queue ensures critical devices (e.g., timer, disk) are serviced before lower-priority ones.

[8-10 Marks]

Q3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.

Approach

A selector function inspects a device's latency tolerance and required bandwidth, then chooses the access mode that better matches those needs before every transfer.

```
BEGIN Select_IO_Mode(device)
    latencyNeed    = device.getLatencyRequirement() // e.g., microseconds
    bandwidthNeed  = device.getBandwidthRequirement() // e.g., MB/s

    IF bandwidthNeed > BANDWIDTH_THRESHOLD
        OR latencyNeed < LATENCY_THRESHOLD THEN
        // High-speed / latency-sensitive: use memory-mapped I/O
        mode = MEMORY_MAPPED_IO
        device.regionAddr = allocateMemoryMappedRegion(device)
        ACCESS = (addr) => CPU.load/store(addr) // ordinary instr.
    ELSE
        // Low-speed / infrequent: use I/O-mapped (isolated) I/O
        mode = IO_MAPPED_IO
        device.portAddr = allocateIOPort(device)
        ACCESS = (port) => CPU.IN/OUT(port) // special instr.
```

```

    END IF

    device.mode = mode
    RETURN mode
END

// Re-evaluated periodically or on workload change
ON WorkloadChange(device):
    newMode = Select_IO_Mode(device)
    IF newMode != device.mode THEN
        reconfigureDeviceAccess(device, newMode)
    END IF

```

This keeps fast, bandwidth-heavy devices (e.g., GPU, NVMe) on the low-overhead memory-mapped path, while simple, infrequent devices use I/O-mapped ports, adapting automatically as requirements change.

[8-10 Marks]

Q4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.

Approach

A central arbiter collects bus requests from all bridge controllers (PCIe, USB, Thunderbolt endpoints) and grants the shared memory/root-complex path using priority plus round-robin fairness within each priority class.

```

requestQueue = [] // {device, busType, priority}

ON BusRequest(device, busType):
    priority = getPriority(busType) // PCIe > Thunderbolt > USB (example)
    requestQueue.add(device, busType, priority)

PROCEDURE BUS_ARBITER():
    LOOP FOREVER
        IF requestQueue is EMPTY THEN WAIT; CONTINUE

        // Pick highest priority class present
        topPriority = min(priority of all entries in requestQueue)
        group = requestQueue.filter(priority == topPriority)

        // Round-robin within the class for fairness
        winner = group.nextInRoundRobin()

        grantBus(winner.device)
        winner.device.transferData()
        releaseBus(winner.device)
        requestQueue.remove(winner)
    END LOOP
END PROCEDURE

PROCEDURE getPriority(busType):
    SWITCH busType:
        CASE PCIe: RETURN 0 // highest - GPU/NVMe traffic
        CASE Thunderbolt: RETURN 1
        CASE USB: RETURN 2 // lowest - tolerant of delay
    END SWITCH
END PROCEDURE

```

Priority levels stop slow USB traffic from blocking latency-critical PCIe transfers, while round-robin inside each class prevents starvation among same-priority devices.

[8-10 Marks]

Q5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Approach

Devices are classified once by speed class. Low-speed devices are checked in a periodic polling loop (simple, low hardware cost); high-speed devices use DMA transfers that raise an interrupt only on completion.

```
BEGIN Hybrid_IO_Manager(deviceList)
    lowSpeedDevices = deviceList.filter(speed < SPEED_THRESHOLD)
    highSpeedDevices = deviceList.filter(speed >= SPEED_THRESHOLD)

    // High-speed devices: set up DMA + interrupt once
    FOR EACH d IN highSpeedDevices DO
        configureDMA(d, d.bufferAddr, d.memAddr, d.size)
        d.onComplete = ISR_HighSpeed_Complete
    END FOR

    // Main polling loop for low-speed devices only
    LOOP FOREVER
        FOR EACH d IN lowSpeedDevices DO
            IF d.statusRegister == DATA_READY THEN
                readData(d)
                d.statusRegister = CLEAR
            END IF
        END FOR
        SLEEP(pollInterval)    // avoid busy-wait wasting CPU
    END LOOP
END

ON INTERRUPT(DMA_COMPLETE) FOR device d:
    ISR_HighSpeed_Complete(d):
        status = DMAC.readStatus(d)
        IF status == OK THEN
            markBufferReady(d.memAddr)
        ELSE
            handleDMAError(d)
        END IF
        DMAC.clearInterrupt(d)
    RETURN FROM INTERRUPT
```

This hybrid design avoids the interrupt-overhead of trivial low-speed devices while giving high-throughput devices zero-poll, low-CPU-overhead DMA transfer — balancing simplicity and performance.

[8-10 Marks]